MODULE 6

Building iOS User Interfaces

Enterprise Mobile Application Development Bootcamp | AI-Integrated Edition

Phase 02 | iOS Development with Swift

Module 6 Roadmap

Seven blocks. By the end: a working, accessible, animated PNC Mobile screen.

1	2	3	4	5	6	7
						
UIKit Fundamentals	**SwiftUI Fundamentals**	**Navigation Patterns**	**Tables & Collections**	**State & Responsive UI**	**Accessibility**	**Gestures & Animation**

UIKit Fundamentals

- Views & the Responder Chain
- Auto Layout
- Stack Views

UIView, View Controllers & the Lifecycle

UIView

The base class for anything drawn on screen - buttons, labels, containers all inherit from it.

UIViewController

Owns one screen's worth of views and manages their lifecycle.

viewDidLoad()

Called once, after the view hierarchy is loaded into memory. Do one-time setup here.

viewWillAppear()

Called every time the screen is about to become visible - refresh data here.

UIView Lifecycle Events

A UIView goes through many events throughout its lifecycle. The runtime will invoke these events, if you define them.

loadView()	viewDidLoad (once)	viewWillAppear (every time)	viewDidAppear (every time)	viewWillDisappear (every time)	viewDidDisappear (every time)

Auto Layout: The Constraint That Gets Forgotten

- This code results in constraints being silently ignored
- It also has Auto Layout warnings in the console

Code Bite

```
let label = UILabel()
label.text = "Balance"
view.addSubview(label)

NSLayoutConstraint.activate([
  label.topAnchor.constraint(
    equalTo: view.safeAreaLayoutGuide.topAnchor,
      constant: 20),
  label.leadingAnchor.constraint(
    equalTo: view.leadingAnchor, constant: 16)
])
```

translateAutoResizingMaskIntoConstraints

- This is a Bool property every UIView has
- It determines whether a view's old-style autoresizing mask is automatically converted into modern Auto Layout constraints
- When true (Default for code), the system automatically generates immutable constraints (of type NSAutoresizingMaskLayoutConstraint) to match the view's current frame and autoresizingMask
  - If you try to add your own programmatic layout constraints on top of this, they will almost always conflict, causing layout crashes or unexpected UI breakages
- When false: The autoresizing mask is ignored.
  - The system stops generating automatic constraints, leaving you with full, explicit control to define the view’s size and position using Auto Layout anchors or constraints

Rule of Thumb

Context	Default Value	What you should do
Programmatic views let v = UIView()	true	Set it to false right before adding your own constraints
Interface Builder (Storyboards and XIBs)	false	Leave it alone (the system flips it to false the moment you add your own constraints in the designer
System-managed views (UITableViewCell, etc.)	true	Leave it alone (changing it on views managed directly by UIKit can break their built-in layout engines)

Auto Layout: The Fix

- The fix is to disable translation of the Autoresizing Mask into Constraints

Code Bite

```
let label = UILabel()
label.text = "Balance"
label.translatesAutoresizingMaskIntoConstraints
  = false
view.addSubview(label)

NSLayoutConstraint.activate([
  label.topAnchor.constraint(
    equalTo: view.safeAreaLayoutGuide.topAnchor,
    constant: 20),
  label.leadingAnchor.constraint(
    equalTo: view.leadingAnchor, constant: 16)
])
```

UIStackView

- Most real screens are just rows and columns of things
- UIStackView handles that layout without needing constraints between every sibling item
- NSLayoutConstraints are only needed on the top-level UIStackView
  - It will automatically lay out its child views
- You can nest them inside of each other
  - E.g. a vertically oriented stack view containing horizontally oriented stack views to create both rows and columns

UIStackView: Auto Layout Without the Boilerplate

```
let stack = UIStackView(arrangedSubviews: [
  balanceLabel, accountNumberLabel, statusIcon
])
stack.axis = .vertical
stack.spacing = 12
stack.alignment = .leading
stack.translatesAutoresizingMaskIntoConstraints = false
view.addSubview(stack)

NSLayoutConstraint.activate([
  stack.topAnchor.constraint(equalTo: view.safeAreaLayoutGuide.topAnchor, constant: 20),
  stack.leadingAnchor.constraint(equalTo: view.leadingAnchor, constant: 16),
  stack.trailingAnchor.constraint(equalTo: view.trailingAnchor, constant: -16)
])
```

Block 1 - UIKit Fundamentals

Skill Check ✓

- What's the one-line fix for a programmatically created view whose constraints appear to do nothing?
- Which lifecycle method runs once, and which runs every time a screen becomes visible?
- When would you reach for UIStackView instead of hand-written NSLayoutConstraints?

SwiftUI Fundamentals

- The View Protocol
- @State & @Binding
- @StateObject, @ObservedObject & @EnvironmentObject

The SwiftUI View Protocol

- A SwiftUI View is not a persistent object the way a UIView is
  - It is a lightweight, disposable description of what the UI should look like right now, given the current state
- SwiftUI recreates these structs constantly as state changes
- That is expected and cheap, not a performance problem to work around
- The View protocol simply requires one property called body
  - It should return a description of what should be onscreen
- Everything you see on screen is a View
  - Text label? - View
  - Button? - View
  - Image? - View
  - Invisible containers for layout? - View

SwiftUI Code

```
struct BalanceCardView: View {
    let accountName: String
    let balance: Decimal

    var body: some View {
        VStack(alignment: .leading, spacing: 8) {
            Text(accountName).font(.headline)
            Text(balance, format: .currency(code: "USD")).font(.largeTitle.bold())
        }
        .padding()
        .background(.thinMaterial)
        .clipShape(RoundedRectangle(cornerRadius: 16))
    }
}
```

@State

- @State creates and owns a piece of data within a view
- The $ prefix (as in $amount) creates a Binding from a @State property
  - It's the mechanism that lets AmountField modify TransferAmountView's amount without owning it itself.
- When a @State value changes, SwiftUI automatically re-renders the owning view and any child views relying on that data

Code Bite

```
struct TransferAmountView: View {
    @State private var amount = ""

    var body: some View {
        AmountField(amount: $amount)
    }
}
```

@Binding

- @Binding references and modifies data owned by another view
- The $ prefix (as in $amount) creates a Binding from a @Binding property
  - It's the mechanism that lets TextField modify AmountField's amount without owning it itself.
- When a child updates a @Binding value, it directly modifies the parent's @State, causing both views to sync and re-render instantly

Code Bite

```
struct AmountField: View {
    @Binding var amount: String

    var body: some View {
        TextField("$0.00", text: $amount)
    }
}
```

@State vs @Binding

Feature	@State	@Binding
Ownership	Owns the data	References external data
Storage	Allocates physical memory for the value	Stores a reference pointer to the data
Initialization	Needs an initial default value	Cannot have a default value
Access Scope	Best marked as private to the current view	Passed in from a parent view
Syntax Usage	Read: variable Write link: $variable	Read: variable Write link: $variable

@StateObject vs. @ObservedObject vs. @EnvironmentObject

Wrapper	Use When
@StateObject	This view CREATES and owns the object. It survives view re-creation
@ObservedObject	This view was HANDED the object by a parent. Does not own its lifecycle
@EnvironmentObject	The object is injected from far up the view tree, shared implicitly

Block 2 - SwiftUI Fundamentals

Skill Check

- A view creates a SessionManager itself. Which property wrapper does it use?
- A child view receives a SessionManager from its parent's initializer. Which wrapper does it use?
- What does the $ prefix on a @State property actually produce?

Navigation Patterns

- UINavigationController
- NavigationStack & NavigationLink
- Tab Bars & Modals

UIKit: UINavigationController

- The window's rootViewController is usually setup in the SceneDelegate
- pushViewController() is used to navigate to a detail screen when you want to be able to return to the current screen
- popViewController() will navigate back to the screen displayed before the call to pushViewController()

Code Bite

```
let nav = UINavigationController(
  rootViewController: AccountListViewController()
)
window?.rootViewController = nav



let detailVC =
AccountDetailViewController(account: account)
navigationController?.pushViewController(detailVC,
animated: true)



navigationController?.popViewController(animated:
true)
```

SwiftUI: NavigationStack & NavigationLink

- Wrap the entire display in a NavigationStack
  - Create a List() out of your array of objects
- Wrap each element's display in a NavigationLink
  - Specify the object as its value property
  - Display content of your choice inside
- Set the navigationDestination on the List
  - Its for property lets you specify the data type being sent to the destination content
  - The closure lets you specify what content to display when an item is clicked

Code Bite

```
var body: some View {
  NavigationStack {
    List(accounts) { account in
      NavigationLink(value: account) {
          AccountRow(account: account)
      }
    }.navigationDestination(for: Account.self) {
      account in
        AccountDetailView(account: account)
  }.navigationTitle("Accounts")
}
```

Tab Bars & Modals

Pattern	UIKit	SwiftUI
Tab Bar	UITabBarController	TabView { . . . }
Modal / Sheet	present(_:animated:)	.sheet(isPresented:) { . . . }
Full-Screen Cover	modalPresentationStyle = .fullScreen	.fullScreenCover(isPresented:) { . . . }

Block 3 - Navigation Patterns

Skill Check

- What SwiftUI modifier connects a NavigationLink's value to the view that should display it?
- Which UIKit method pushes a new screen onto the navigation stack?
- When would you reach for a full-screen cover instead of a sheet?

Table Views & Collection Views

- UITableView: DataSource & Delegate
- SwiftUI List
- UICollectionView & Diffable Data Sources

UITableView

- The UITableView is responsible for displaying a list of cells
- Each cell is created to display UI based on an element in a collection of data
- It efficiently recycles cells as they are scrolled offscreen
  - Offscreen cells are re-used as the cells that are being scrolled onscreen

Code Bite

```
let tableView = UITableView()


self.view.addSubview(tableView)

tableView.translatesAutoresizingMaskIntoCons
traints = false


NSLayoutConstraint.activate([

  tableView.topAnchor.constraint(. . . ),

  . . .

])
```

UITableViewDataSource Protocol

- This protocol provides the data and rendering methods for the UITableView
  - It determines "what goes into each row"
- One method returns the total number of data points in the collection
- The other method configures a single cell to display the content you wish

Code Bite

```
class MyClass: UITableViewDataSource {
  func tableView(_ tableView: UITableView,
    numberOfRowsInSection section: Int) ->
      Int {
        return myDataColl.count
  }
  func tableView(_ tableView: UITableView,
    cellForRowAt indexPath: IndexPath) ->
      UITableViewCell {
        // get the cell and set its content
  }
}
```

Configuring Content

- dequeueReusableCell will get an available cell to recycle or create a new one, as necessary
- The identifier used ("cell" in this case) is specified in the configuration of the UITableView using its register() method
- You can also create your own custom cell class for specialized content

Code Bite

```
// in the cellForRowAt method:
let cell = tableView.dequeueReusableCell(
   withIdentifier: "cell", for: indexPath)

cell.textLabel?.text = "some content"

return cell
```

UITableViewDelegate Protocol

- This protocol provides methods to react to user interactions with a row
  - It determines "what the row does"
- Actions include:
  - Selecting a row
  - Deselecting a row
  - Preventing selection of a row
  - Highlighting when user touches a row
  - Leading or trailing swipe actions
  - Dragging for reordering

Code Bite

```
class MyClass: UITableViewDelegate {
  func tableView(_ tableView: UITableView,
    didSelectRowAt indexPath: IndexPath) {
      . . .
  }

  func tableView(_ tableView: UITableView,
    trailingSwipeActionsConfigurationForRowAt
    indexPath: IndexPath) {
      . . .
  }
}
```

UIKit: UITableView DataSource & Delegate

```
func tableView(_ tv: UITableView, numberOfRowsInSection s: Int) -> Int {
    accounts.count
}

func tableView(_ tv: UITableView, cellForRowAt ip: IndexPath)
  -> UITableViewCell {
    let cell = tv.dequeueReusableCell(withIdentifier: "AccountCell", for: ip) as! AccountCell
    cell.configure(with: accounts[ip.row])
    return cell
}

func tableView(_ tv: UITableView, didSelectRowAt ip: IndexPath) {
    let detail = AccountDetailViewController(account: accounts[ip.row])
    navigationController?.pushViewController(detail, animated: true)
}
```

SwiftUI: List - the Table View Replacement

```
List(accounts) { account in
    HStack {
        VStack(alignment: .leading) {
            Text(account.name).font(.headline)
            Text(account.number).font(.caption)
                .foregroundStyle(.secondary)
        }
        Spacer()
        Text(account.balance, format: .currency(code: "USD"))
            .font(.body.monospacedDigit())
    }
    .accessibilityElement(children: .combine)
}
.listStyle(.insetGrouped)
```

UICollectionView & Diffable Data Sources

UICollectionView

Grid-capable sibling of UITableView - flexible layouts via UICollectionViewLayout.

Compositional Layout

Modern, declarative way to describe sections, groups, and items.

Diffable Data Source

Populate with a snapshot; the framework computes the minimal diff and animates it - no manual reloadData bookkeeping.

Block 4 - Tables & Collections

Skill Check ✓

- In UIKit, which protocol answers 'what goes in each row,' and which answers 'what happens on tap'?
- What single SwiftUI view replaces UITableView's DataSource, Delegate, and cell reuse entirely?
- What problem does a diffable data source solve that manual reloadData calls don't?

State Management & Responsive Layouts

- Single Source of Truth
- Size Classes & Trait Collections
- Dynamic Type

State Management: Single Source of Truth

If two views can disagree about what the current balance is, the state doesn't have a single source of truth.

- Practical rule: state flows down through the view hierarchy as parameters or Bindings; events flow up as function calls or closures. Views should not reach sideways to mutate a sibling's state directly.

Responsive Layouts: Size Classes & Trait Collections

Compact width

iPhone portrait, iPhone landscape (most models)

Regular width

iPad, iPhone Pro Max landscape, iPad split view (wider pane)

Code Bite

```
@Environment(\.horizontalSizeClass) private
var sizeClass


var body: some View {
  if sizeClass == .regular {
    HStack {              // iPad: side-by-side
      AccountListView();
      AccountDetailView()
      }
    } else {
      NavigationStack {     // iPhone: stack
        AccountListView() }
      }
}
```

Dynamic Type & Adaptive Stacks

```
// Avoid this
Text("Available Balance")
    .font(.system(size: 17))
    // fixed size - ignores the user's text-size preference entirely

// Prefer this
Text("Available Balance")
    .font(.headline)   // scales with Dynamic Type

ViewThatFits {
    HStack { balanceLabel; statusIcon }
    VStack { balanceLabel; statusIcon }
}
```

Block 5 - State & Responsive Layouts

Skill Check ✓

- What's the practical symptom of state that doesn't have a single source of truth?
- What size class would an iPad in full-screen (non-split) typically report for its horizontal axis?
- Why does .font(.system(size: 17)) fail participants who rely on larger system text sizes?

Accessibility Fundamentals

- VoiceOver
- Accessibility Labels & Traits
- the Accessibility Inspector

VoiceOver

- A screen that looks correct can still be unusable if VoiceOver reads it as “button, button, image” with no context.

Code Bite

```
Text(account.balance, format:
.currency(code: "USD"))
    .accessibilityLabel("Available balance")
    .accessibilityValue("\(account.balance)
dollars")
```

Accessibility Labels, Traits & the Inspector

.accessibilityLabel()

What VoiceOver reads aloud for this element

.accessibilityTraits()

The element's role - .isButton, .isHeader, .isSelected

Accessibility Inspector

Xcode's dedicated tool (Xcode > Open Developer Tool) for auditing labels, contrast, and Dynamic Type live

Block 6 - Accessibility Fundamentals

Skill Check

- What's the difference between what accessibilityLabel and accessibilityTraits each control?
- Why can a screen that looks visually correct still fail an accessibility review?
- What Xcode tool would you use to audit a screen's accessibility before shipping it?

Gesture Handling & Animation Basics

- UIKit Gesture Recognizers
- SwiftUI Gestures
- UIView.animate & withAnimation

Gesture Recognizers in UIKit

Class	Type	Use Case	Key Properties
UITapGestureRecognizer	Discrete	Buttons, image selection, dismissals	numberOfTapsRequired, numberOfTouchesRequired
UIPanGestureRecognizer	Continuous	Dragging items, sliding drawers	minimumNumberOfTouches, maximumNumberOfTouches
UIPinchGestureRecognizer	Continuous	Zooming in/out	scale, velocity
UIRotationGestureRecognizer	Continuous	Rotating components or images	rotation, velocity
UISwipeGestureRecognizer	Discrete	Changing views or sliding items out of view	direction
UILongPressGestureRecognizer	Continuous	Displaying context menus, entering edit modes	minimumPressDuration, allowableMovement
UIScreenEdgePanGestureRecognizer	Continuous	Edge navigation swipes	edges

Gesture Recognizers: UIKit

- The various gesture recognizers were written for Objective-C
  - And have not been updated
- Their target property identifies the object whose method should be called when the gesture is recognized
- Their selector property specifies which function of the object should be executed
- The function must be exposed to objective-c
  - By default, all functions are not
  - The @objc wrapper will expose the function

Code Bite

```
let tap = UITapGestureRecognizer(
  target: self, action:
#selector(cardTapped)
)
cardView.addGestureRecognizer(tap)
cardView.isUserInteractionEnabled = true


@objc func cardTapped() {
    showDetail()
}
```

Gesture Recognizers in SwiftUI

- SwiftUI has 5 recognizers built into the framework and invoked with a modiefier

Gesture Struct	Convenience Modifier
TapGesture	.onTapGesture(count: perform:)
LongPressGesture	.onLongPressGesture(minimumDuration: maximumDistance: perform: onPressingChanged:)
DragGesture	.gesture(DragGesture())
MagnificationGesture/PinchGesture	.gesture(MagnificationGesture())
RotationGesture	.gesture(RotationGesture())

Gesture Recognizers: SwiftUI

- You can attach a gesture to a view using either a specialized convenience modifier or the generic .gesture() modifier

Code Bite

```
BalanceCardView(account: account)
    .onTapGesture {
        isScaled.toggle()
    }
    .onTapGesture(count: 2) {
        showDetail = true
    }
```

Animation Basics - UIKit

- Animations in UIKit allow you to smoothly change view properties over time
- Closure blocks are the most common way to animate
  - Modify a view's animatable properties within the closure block and UIKit handles the transition

Code Bite

```
UIView.animate(withDuration: 0.3) {
    self.cardView.alpha = 0
    self.cardView.transform =
      CGAffineTransform(
        translationX: 0, y: -20
      )
}
```

Animation Basics - SwiftUI

- Animation in SwiftUI is driven entirely by state changes
- Explicit animation is declared where the state actually changes
  - withAnimation wraps a state change
  - SwiftUI will animate every view property that depends on that specific state change
  - Even in descendant views
- Implicit animation is attached to a view modifier
  - .animation(value:) will affect only the animatable properties above it in the modifier chain

Code Bite

```
// Explicit animation
withAnimation(.easeInOut(duration: 0.3)) {
    isExpanded.toggle()
}

// Implicit animation
Button("Tap Me") {
  isScaled.toggle()
}
.padding(50)
.scaleEffect(isScaled ? 1.5 : 1.0)
.animation(.spring(), value: isScaled)
```

Block 7 - Gestures & Animation

Skill Check

- In UIKit, what two things does a view need for a tap gesture to actually fire?
- What's the difference between wrapping a change in withAnimation and attaching .animation(value:)?

PNC Mobile: Accounts List Screen

Skill Check

- Build a SwiftUI AccountListView using List, NavigationStack, and NavigationLink.
- Each row shows account name, masked account number, and balance (currency-formatted).
- Tapping a row navigates to an AccountDetailView showing the full account.
- Every row must be fully readable by VoiceOver as one combined element, not three separate announcements.
- The balance text must scale correctly under larger Dynamic Type sizes - no fixed font sizes.

Module 6 Complete

You can now build screens in both UIKit and SwiftUI, wire up navigation, populate lists and grids, manage state with a single source of truth, adapt layouts across device sizes, make screens accessible, and add gestures and animation. Module 7 builds the architecture that organizes screens like these at real application scale.

Next: Module 7 - iOS Application Architecture